Name: Pranav Pol Class : D20A Roll no: 44 Batch: C

EXPERIMENT 6

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

THEORY:

A **Smart Contract** is a self-executing digital program stored on a blockchain that automatically carries out the terms of an agreement when specific conditions are fulfilled. It removes the need for intermediaries, allowing parties to interact directly while maintaining transparency, security, and reliability. Smart contracts are typically written using **Solidity** for blockchain platforms like **Ethereum**, and they run on the **Ethereum Virtual Machine**, which executes the contract code across the network.

Significance of Smart Contracts in the Ethereum Blockchain

Automation:
Smart contracts automatically execute predefined actions once the specified conditions are satisfied, reducing manual intervention.

Transparency:
All contract code and related transactions are recorded on the blockchain, making them publicly visible and verifiable.

Security:
After deployment, smart contracts cannot be easily altered, making them resistant to tampering and fraud.

Cost Efficiency:
By removing intermediaries such as brokers or third-party services, smart contracts help reduce operational and transaction costs.

Trustless Interaction:
Participants can engage in transactions without needing to rely on trust, as the contract logic itself ensures fair execution.

Smart contracts serve as the foundation for many blockchain-based applications, including decentralized applications (DApps) and decentralized finance (DeFi) systems built on Ethereum.

Implementation

Step 1: Open Remix IDE

1. Open your browser.
2. Navigate to **Remix IDE**.
3. Create a new file named:

Escrow.sol

Step 2:Write the Smart Contract Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Escrow {

    address public buyer;
    address payable public seller;
    address public arbiter;

    bool public buyerApproved;
    bool public sellerApproved;

    constructor(address payable _seller, address _arbiter) payable {
        buyer = msg.sender;
        seller = _seller;
        arbiter = _arbiter;
    }

    function approve() public {
        require(msg.sender == buyer || msg.sender == seller, "Not authorized");

        if (msg.sender == buyer) {
            buyerApproved = true;
        }

        if (msg.sender == seller) {
            sellerApproved = true;
        }

        if (buyerApproved && sellerApproved) {
```

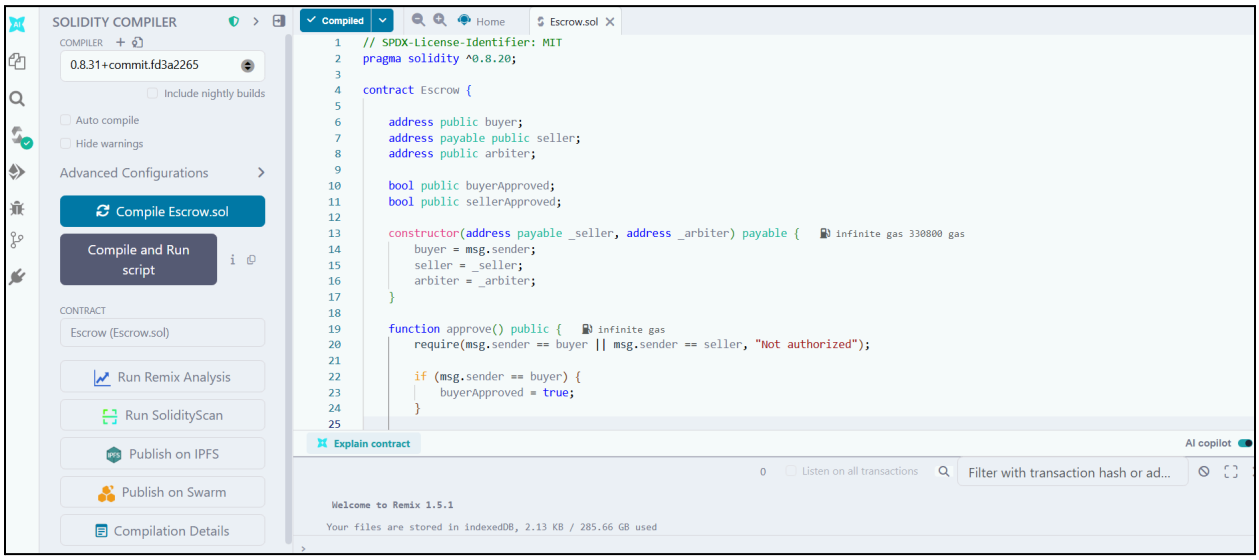
```
        uint amount = address(this).balance;

        (bool success, ) = seller.call{value: amount}("");

        require(success, "Transfer failed");
    }
  }
}
```

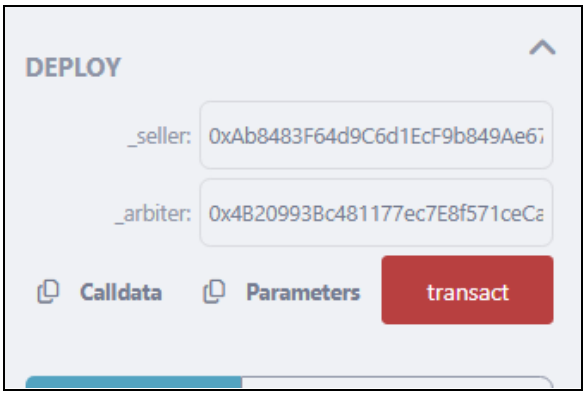
Step 3: Compile the Contract

1. Click the **Solidity Compiler** tab.
2. Select version **0.8.x**.
3. Click **Compile Escrow.sol**.

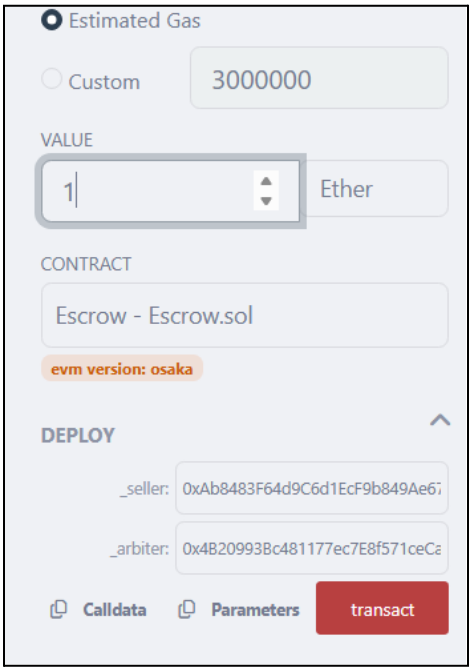


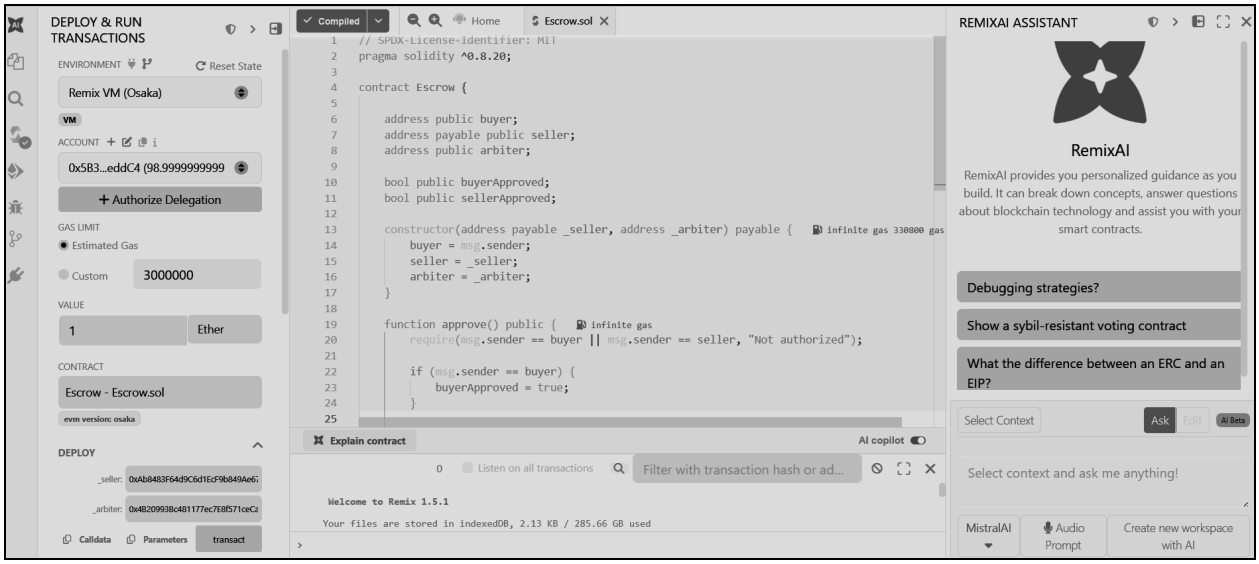
Step 4: Deploy the Contract

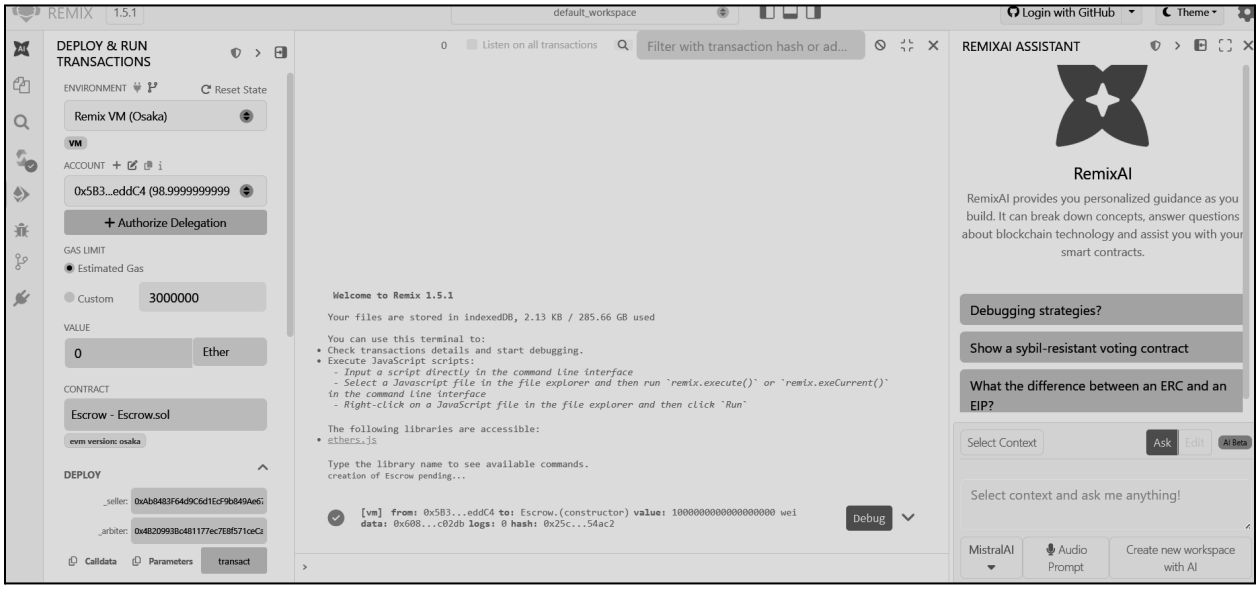
1. Open **Deploy & Run Transactions**.
2. Select environment **Remix VM**.
3. Enter seller and arbiter addresses.

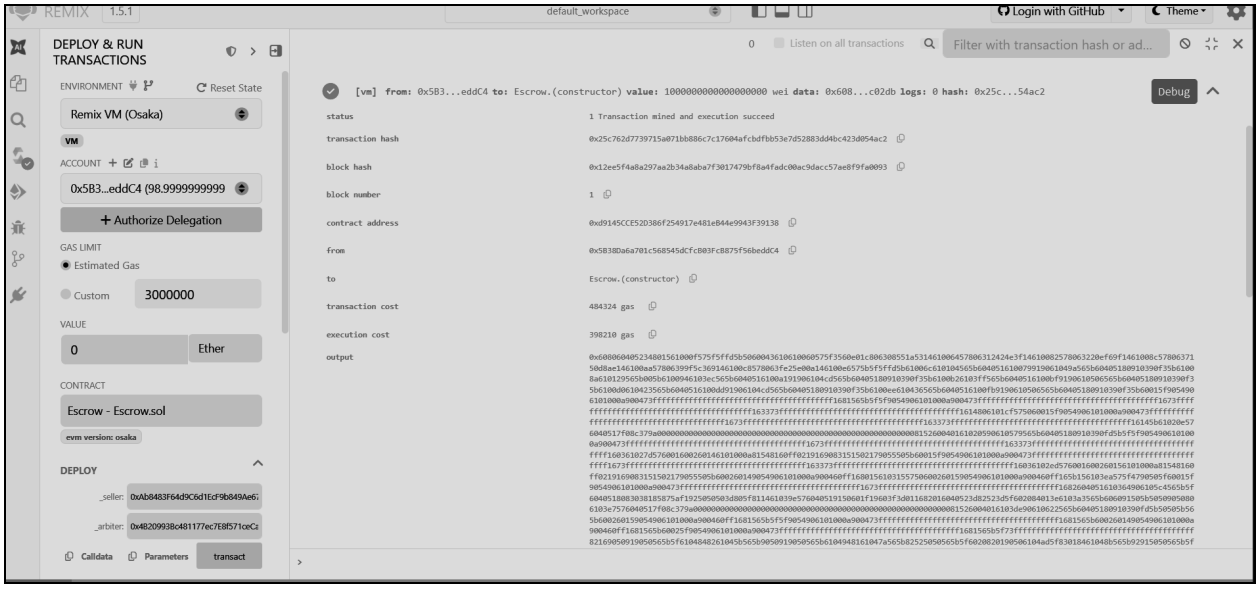


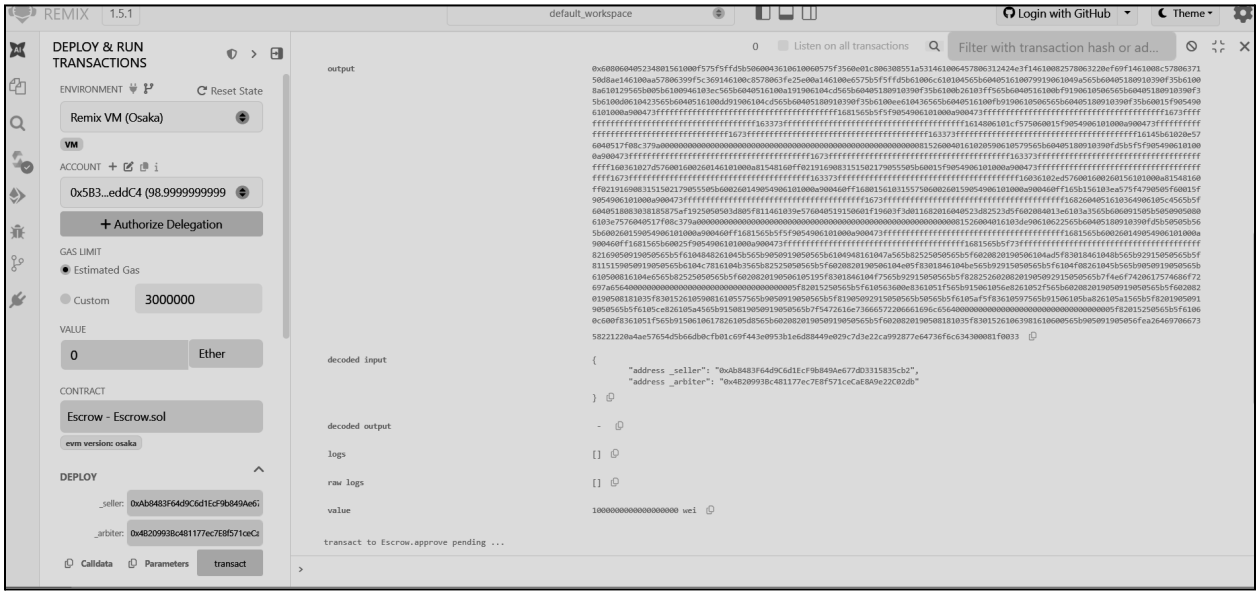
4. Deploy the contract with some **Ether value**.




DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
Reset State
Remix VM (Osaka)
VM
ACCOUNT
0x5B3...eddC4 (98.9999999999
+ Authorize Delegation
GAS LIMIT
Estimated Gas
Custom
3000000
VALUE
1
Ether
CONTRACT
Escrow - Escrow.sol
evm version: osaka
DEPLOY
_seller: 0xAb8483F64d9C6d1EcF9b849Ae67
_arbiter: 0x4B20993Bc481177ec7E8f571ceCa
Calldata
Parameters
transact
Compiled
Home
Escrow.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract Escrow {
address public buyer;
address payable public seller;
address public arbiter;
bool public buyerApproved;
bool public sellerApproved;
constructor(address payable _seller, address _arbiter) payable {
infinite gas 330000 gas
buyer = msg.sender;
seller = _seller;
arbiter = _arbiter;
}
function approve() public {
infinite gas
require(msg.sender == buyer || msg.sender == seller, "Not authorized");
if (msg.sender == buyer) {
buyerApproved = true;
}
Explain contract
AI copilot
0
Listen on all transactions
Filter with transaction hash or ad...
Welcome to Remix 1.5.1
Your files are stored in indexedDB, 2.13 KB / 285.66 GB used
REMIXAI ASSISTANT
RemixAI
RemixAI provides you personalized guidance as you build. It can break down concepts, answer questions about blockchain technology and assist you with your smart contracts.
Debugging strategies?
Show a sybil-resistant voting contract
What the difference between an ERC and an EIP?
Select Context
Ask
Edit
AI Beta
Select context and ask me anything!
MistralAI
Audio Prompt
Create new workspace with AI


REMIX 1.5.1
default_workspace
Login with GitHub
Theme
DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
Reset State
Remix VM (Osaka)
VM
ACCOUNT
0x5B3...eddC4 (98.9999999999
+ Authorize Delegation
GAS LIMIT
Estimated Gas
Custom
3000000
VALUE
0
Ether
CONTRACT
Escrow - Escrow.sol
evm version: osaka
DEPLOY
_seller: 0xAb8483F64d9C6d1EcF9b849Ae67
_arbiter: 0x4B20993Bc481177ec7E8f571ceCa
Calldata
Parameters
transact
0
Listen on all transactions
Filter with transaction hash or ad...
Welcome to Remix 1.5.1
Your files are stored in indexedDB, 2.13 KB / 285.66 GB used
You can use this terminal to:
• Check transactions details and start debugging.
• Execute JavaScript scripts:
- Input a script directly in the command line interface
- Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.exeCurrent()` in the command line interface
- Right-click on a JavaScript file in the file explorer and then click `Run`
The following libraries are accessible:
• ethers.js
Type the library name to see available commands.
creation of Escrow pending...
[vm] from: 0x5B3...eddC4 to: Escrow.(constructor) value: 100000000000000000 wei
data: 0x608...c02db logs: 0 hash: 0x25c...54ac2
Debug
REMIXAI ASSISTANT
RemixAI
RemixAI provides you personalized guidance as you build. It can break down concepts, answer questions about blockchain technology and assist you with your smart contracts.
Debugging strategies?
Show a sybil-resistant voting contract
What the difference between an ERC and an EIP?
Select Context
Ask
Edit
AI Beta
Select context and ask me anything!
MistralAI
Audio Prompt
Create new workspace with AI



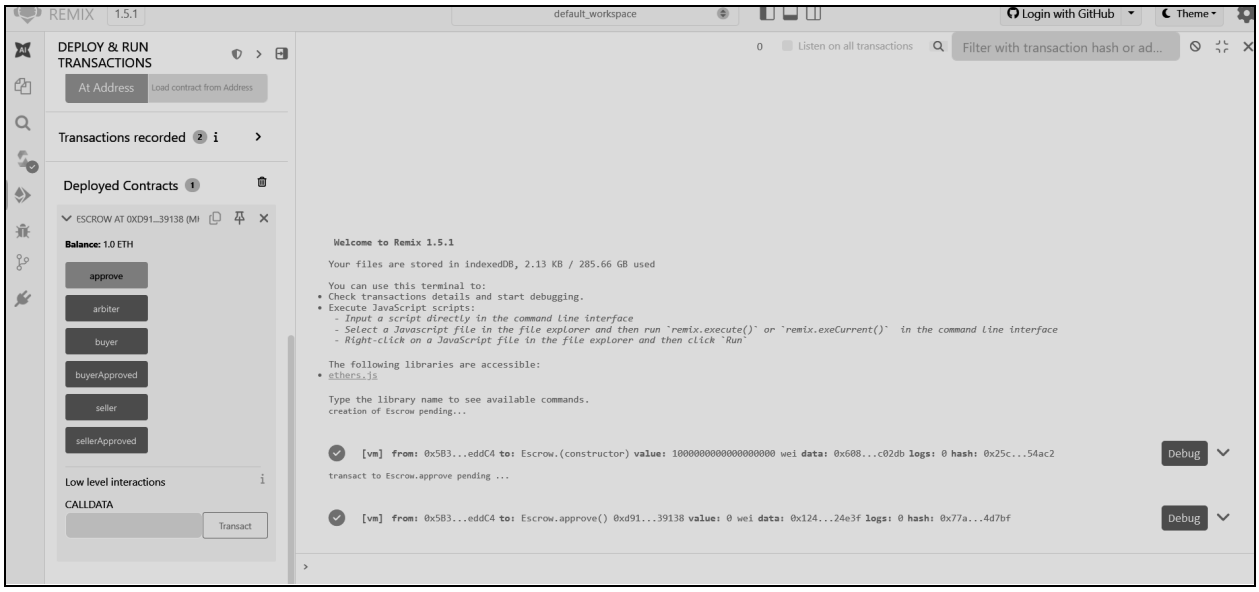


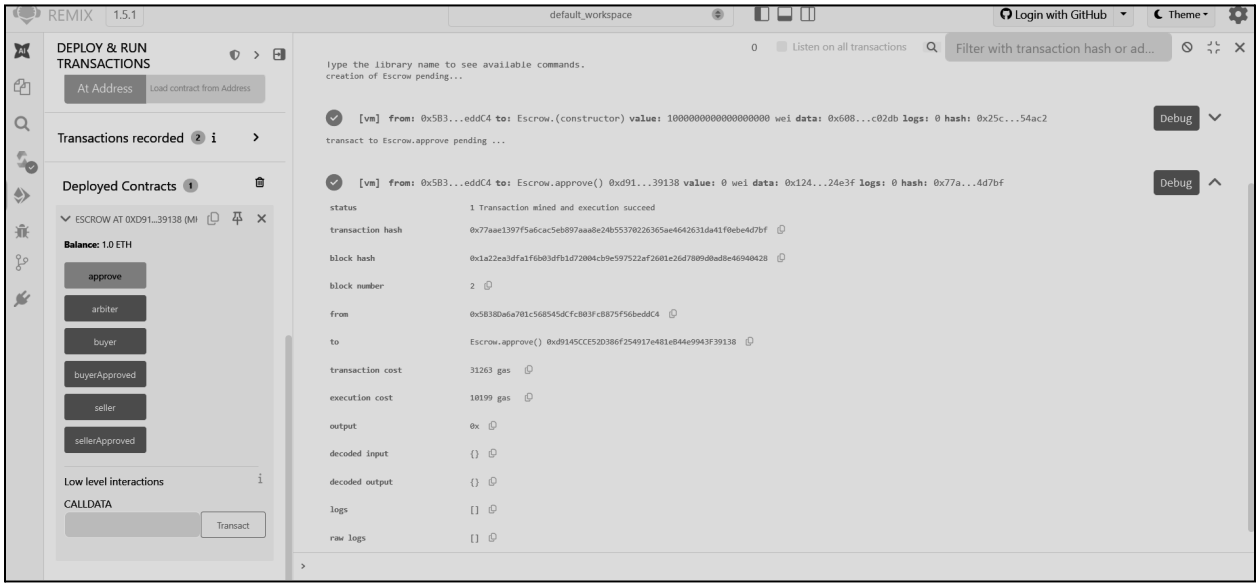
Step 5: Execute Transactions

1. Buyer deposits Ether while deploying.
2. Seller and buyer both call the **approve()** function.
3. Once both approve, the smart contract automatically transfers funds to the seller.

Buyer Approval

1. Keep **Account 1 selected**
2. Click: approve()

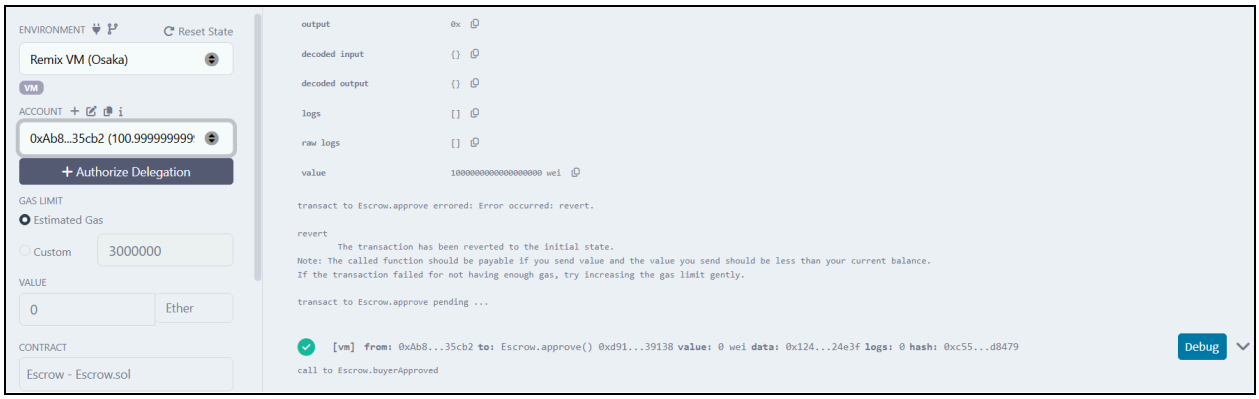




Seller Approval

1. Change **Account → Seller account**
2. Click: approve()

Now the **smart contract sends ETH to the seller automatically**.



The smart contract successfully:

- Held funds securely in escrow.
- Allowed buyer and seller to approve the transaction.
- Released funds automatically when both parties approved.

Transaction details such as **gas used**, **transaction hash**, and **block information** were generated.

CONCLUSION:

This experiment demonstrated the creation and deployment of a decentralized escrow smart contract using **Solidity** in the **Remix IDE** environment. The smart contract was successfully compiled, deployed, and executed on the **Ethereum Virtual Machine**.
Through this experiment, it was observed that the escrow contract securely holds funds and releases them only when predefined conditions are satisfied by both parties. This demonstrates how blockchain technology can automate transactions without relying on intermediaries. The experiment also helped in understanding the working of smart contracts, deployment process, and interaction with blockchain-based applications, highlighting their potential for secure and transparent digital transactions.